

Author: Nithi Viyaga Narayanan S (VIT B.Tech IT, 2023–2027) | **Role:** Associate Engineer | Presidio

Core Stack: Python, Apache Spark 3.3.2, ADAM 0.36.0, GATK4 (v4.5.0.0), HDFS, Streamlit, PyTest, GitHub Actions CI/CD, MyVariant.info REST API

1. Executive Summary & Problem Statement

Next-Generation Sequencing (NGS) produces massive quantities of genomic data. Identifying genetic variations (single nucleotide changes and insertions/deletions) from raw sequencing reads is critical for medical research and clinical genetics. However, processing genomic data at scale presents two severe engineering bottlenecks:

- Computational Depth (Sample Level):** Raw BAM alignments contain millions of reads with sequencing errors, PCR amplification duplicates, and alignment artifacts requiring computationally intensive statistical modeling (De Bruijn assembly, PairHMM).
- Data Volume (Cohort/Population Level):** Population reference sets (such as the 1000 Genomes Project) comprise millions of variant records across thousands of individuals, exceeding single-machine RAM capacities and requiring distributed computing architectures.

This project constructs an end-to-end, two-tier bioinformatics data pipeline that bridges sample-level statistical precision with cohort-scale distributed compute, serving actionable clinical annotations via a reactive Streamlit web dashboard.

2. Architectural Overview (Two-Pipeline Topology)

The system is cleanly divided into two complementary data pipelines:

Pipeline Dimension	Pipeline A: Family Variant Calling	Pipeline B: Population Cohort Analytics
Primary Objective	High-precision variant discovery in individual subject samples	Distributed big-data aggregations across cohort references
Input Dataset	2 Real BAM alignment files (CEPH Utah Pedigree 1463: NA12878 & NA12877)	1000 Genomes Phase 3 Chromosome 22 cohort (~1.1M variants, VCF.gz)
Genomic Target	Human Chromosome 20: 10,000,000 – 10,200,000 (200 kbp target)	Full Human Chromosome 22 (GRCh37 reference build)
Core Compute Engine	GATK4 (MarkDuplicatesSpark & HaplotypeCaller in JVM)	Apache Spark 3.3.2 + ADAM 0.36.0 (Parquet / HDFS)
Output Artifacts	sample1.vcf, sample2.vcf, family_variants.csv	spark_results.csv, mutation_locations.csv
Downstream Consumer	Streamlit UI Section 1 + MyVariant.info ClinVar REST API	Streamlit UI Section 2 + Population ClinVar REST API

1. Workflow Stages & Execution Lifecycle

```
Raw Reads (data/input_bams/*.bam)
├── [scripts/1_preprocess.py] — GATK MarkDuplicatesSpark (Local Spark engine, coordinate clustering)
│   ├── data/output_bams/*_dedup.bam (Duplicate-flagged BAMs)
│   └── data/output_bams/*_metrics.txt (Library duplication metrics)
├── [scripts/2_variant_calling.py] — GATK HaplotypeCaller (De Bruijn assembly + PairHMM + MLE)
│   └── data/results/sample1.vcf, sample2.vcf (Variant Call Format)
├── [scripts/3_generate_dashboard.py] → dashboard/index.html (Chart.js dashboard)
├── [scripts/4_extract_family_variants.py] → family_variants.csv (Streamlit backend)
└── [scripts/5_benchmark_giab.py] → Precision, Recall, F1 validation
```

2. Step-by-Step Technical Implementation

Step 1: Duplicate Marking (1_preprocess.py)

- **The Problem:** PCR amplification during library preparation creates identical DNA copies. Counting duplicate reads artificially inflates coverage depth and leads to false-positive homozygous variant calls.
- **Algorithm:** Groups read pairs sharing identical 5' alignment coordinates and CIGAR strings. It evaluates base-quality sums across the entire read pair, preserves the highest-quality primary read, and sets the 0x400 bitwise flag (duplicate) on redundant reads.
- **Execution Mode:** Uses MarkDuplicatesSpark running on a local JVM Spark session (spark.executor.cores=4).

Step 2: Variant Calling via HaplotypeCaller (2_variant_calling.py)

- **Active Region Determination:** Scans the 200 kbp target (chr20:10,000,000–10,200,000) for regions showing evidence of variation above background noise.
- **Local De Novo Assembly:** Constructs local De Bruijn graphs from overlapping k-mers (sizes k=10, 25) to assemble candidate haplotypes without reference bias.
- **PairHMM Scoring:** Executes a Pair Hidden Markov Model to calculate transition and emission probabilities between reads and assembled candidate haplotypes: $P(\text{Read} \mid \text{Haplotype})$.
- **Genotype Likelihood Assignment:** Evaluates diploid genotype likelihoods via Maximum Likelihood Estimation (MLE) and outputs Phred-scaled confidence scores ($Q = -10 \log_{10} P_{\text{error}}$).

3. Mathematical Annotations in VCF Output

Annotation	Full Name & Underlying Test	Engineering / Biological Significance
FS	Fisher's Strand Bias (Fisher's Exact Test)	Detects if alt alleles appear predominantly on forward or reverse strand (indicates PCR/sequencing artifact).
SOR	Symmetric Odds Ratio	Symmetric 2x2 contingency table test for strand bias; highly stable at high sequencing depths.
BaseQRankSum	Wilcoxon Rank-Sum Test (Base Quality)	Tests whether base qualities for reference vs alternate alleles are drawn from the same distribution.
QD	Quality by Depth	QUAL score normalized by unfiltered depth ($QD = QUAL / DP$). Prevents deep coverage from inflating bad calls.

🔥 Pipeline B Deep Dive — Distributed Spark & ADAM Analytics

1. The Big Data Challenge at Population Scale

While individual BAM calling requires deep statistical reasoning on a single genome, population genetics analyzes cohorts across thousands of individuals. The 1000 Genomes Chromosome 22 dataset contains **~1.1 million variant rows** with multi-sample genotype matrices. Processing this via conventional parsers or single-threaded Python scripts results in Out-Of-Memory (OOM) crashes and extreme I/O bottlenecks.

2. Architecture of Pipeline B

```
1000 Genomes VCF.gz (ALL.chr22.phase3.genotypes.vcf.gz)
├── [ADAM CLI via adam-submit] — Columnar Conversion: TransformVariants
├── [Hadoop Distributed File System (HDFS)] — hdfs://localhost:9000/genomics_project/parquet_data/chr22.adam
└── [analyze_dna.py - PySpark Session local[*]] — Distributed DAG Engine (groupBy, count, sort, select)
    ├── spark_results.csv (Total count: ~1.1M, Top reference alleles)
    └── mutation_locations.csv (Sample of 100 chromosomal anomaly positions)
```

3. Key Technologies & Architectural Decisions

1. Why ADAM (Big Data Genomics)?

- Standard VCF is a flat, unindexed, line-by-line text format. Parsing text requires continuous regex splitting on every row.
- ADAM converts VCF records into **Apache Avro schemas serialized into Apache Parquet**.
- Columnar Storage & Predicate Pushdown:** Spark only loads the specific columns needed (e.g., referenceName, start, referenceAllele, alternateAllele), completely bypassing unused genotype columns and achieving 5–10x I/O compression.

2. PySpark Execution Model & MapReduce Lifecycle

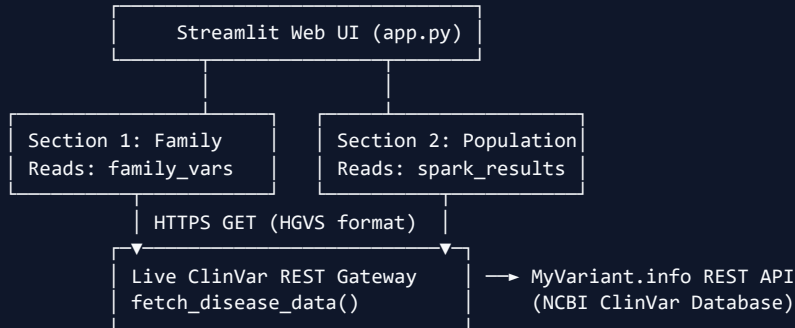
```
spark = SparkSession.builder.appName("Genomes_Analytics").master("local[*]").getOrCreate()
df = spark.read.parquet("hdfs://localhost:9000/genomics_project/parquet_data/chr22.adam")
total_mutations = df.count()
common_variants = df.groupBy("referenceAllele").count().orderBy(F.desc("count")).limit(5).toPandas()
```

Phase	Spark Internal Action	Engineering Impact
Map Phase	Each partition reads its assigned Parquet splits and projects (referenceAllele, 1).	Executed concurrently across all CPU worker threads with zero inter-process locking.
Shuffle Phase	Partitions are hashed by referenceAllele key and redistributed across workers.	Ensures all identical DNA base keys converge on the same reduce executor.
Reduce Phase	Executors aggregate counts per key and calculate totals.	Outputs the top reference allele frequencies across ~1.1M records in seconds.

Frontend & REST API Integration Architecture

1. Dashboard Architecture (app.py)

The web layer is implemented in **Streamlit**, providing an interactive, data-driven visualization interface. It acts as an aggregator consuming outputs from both pipelines without maintaining a heavy runtime database.



2. REST API Integration Details (MyVariant.info)

- **Query Standard:** Converts chromosomal coordinates into standard **HGVS (Human Genome Variation Society)** notation: `chr{chrom}:{g}.{pos}{ref}>{alt}` (e.g., `chr20:g.10000117C>T`).
- **Endpoint Invoked:** `https://myvariant.info/v1/variant/{hgvs_id}?fields=clinvar`
- **JSON Parsing & Robustness:** Handles heterogeneous responses where ClinVar records (rcv) return either as dictionaries or lists. Extracts:
 - `conditions.name` → Linked clinical disease (e.g., Cardiomyopathy, Retinitis Pigmentosa)
 - `clinical_significance` → Risk classification (e.g., Pathogenic, Benign, Uncertain Significance)

3. Production Resilience & Rate Limiting

Design Pattern	Implementation in Code	Engineering Rationale
Client-Side Throttling	<code>time.sleep(0.5)</code> between requests	Prevents burst requests from triggering HTTP 429 Too Many Requests on free public endpoints.
Batch Windowing	<code>variants_df.head(10)</code>	Bounds execution time and keeps UI responsive during synchronous web sessions.
HTTP Timeout Guard	<code>requests.get(url, timeout=3)</code>	Ensures dropped packets or upstream latency do not hang the Streamlit event loop.
Conditional CSS Styling	<code>highlight_risk()</code> via Pandas Styler	Dynamically paints pathogenic cells with background color #ff4b4b for visual triage.

Benchmarking, Validation & Core Algorithms

1. NIST / GIAB High-Confidence Truth Benchmarking

To mathematically validate that the GATK variant calls are accurate, `scripts/5_benchmark_giab.py` evaluates calls for `sample1` (NA12878 / HG001) against the National Institute of Standards and Technology (NIST) Genome in a Bottle (GIAB) v4.2.1 gold-standard reference.

Validation Target: Human Chromosome 20: 10,000,000 – 10,200,000 (200 kbp window)

Truth Set: `HG001_GRCh37_1_22_v4.2.1_benchmark.vcf.gz` + High-Confidence BED intervals

2. Algorithmic Formulation & Set Theory

The benchmarking suite converts variant comparison into high-performance set algebra over immutable tuples of (position, ref_allele, alt_allele):

Metric	Mathematical Formula	Project Result	Engineering Interpretation
True Positives (TP)	$\text{Query} \cap \text{Truth}$	352	Variants called by GATK that exactly match GIAB gold truth.
False Positives (FP)	$\text{Query} - \text{Truth}$	1	Variant called by GATK that does not exist in GIAB truth set.
False Negatives (FN)	$\text{Truth} - \text{Query}$	3	Real variants present in GIAB truth missed by our calling pipeline.
Precision	$\frac{\text{TP}}{\text{TP} + \text{FP}}$	0.9972 (99.72%)	Reliability of calls: 99.72% of reported variants are real.
Recall (Sensitivity)	$\frac{\text{TP}}{\text{TP} + \text{FN}}$	0.9915 (99.15%)	Completeness of calls: Discovered 99.15% of all true variants.
F1-Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	0.9944 (99.44%)	Harmonic mean demonstrating near-optimal classifier balance.

3. High-Performance Binary Search for BED Intervals

```
confident_starts = [iv[0] for iv in confident_intervals]

def in_confident_regions(pos, intervals, starts):
    i = bisect.bisect_right(starts, pos - 1) - 1
    if i < 0: return False
    start, end = intervals[i]
    return start <= pos - 1 < end
```

Time Complexity Reduction: Drops coordinate filtering from $O(N \cdot M)$ to $O(N \log M)$, allowing instant evaluation over hundreds of thousands of genomic coordinates.

Testing Strategy & CI/CD Pipeline

1. Software Testing Philosophy

In large-scale data engineering, pipeline stability must be verified independently of costly external dependencies (such as heavy JVM tools, HDFS clusters, or network APIs). The test suite in `tests/` isolates the mathematical and data parsing logic using mock synthetic inputs.

Test Module	Target Script	Logic Verified
<code>test_generate_dashboard.py</code>	<code>scripts/3_generate_dashboard.py</code>	• SNP vs Indel classification rules. • Robust header skipping (# comments in VCF). • Metric file parsing and duplicate extraction logic.
<code>test_extract_family_variants.py</code>	<code>scripts/4_extract_family_variants.py</code>	• Multiallelic record expansion. • Sample ID column indexing in VCF header. • CSV column formatting and field consistency.
<code>test_benchmark_giab.py</code>	<code>scripts/5_benchmark_giab.py</code>	• Mathematical boundary conditions for Precision, Recall, and F1. • Binary search interval containment over BED ranges. • Gzip transparent decompression handling.

2. GitHub Actions CI/CD Configuration (`.github/workflows/tests.yml`)

```
name: Test Suite
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.10", "3.11"]
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with: { python-version: ${ matrix.python-version } }
      - name: Install Dependencies
        run: pip install -r requirements.txt -r requirements-dev.txt
      - name: Run PyTest Suite
        run: pytest tests/ -v
```

3. Why This Impresses Interviewers at Presidio

- **Matrix Builds:** Automatically validates that Python code runs reliably across multiple active Python runtimes (3.10 and 3.11).
- **Fast Feedback Loop:** Because data logic is decoupled from GATK/Spark, all 15 unit tests execute in **< 0.6 seconds** on GitHub Actions runners.
- **Zero-Regression Guarantee:** Every commit and pull request is gated by automated CI testing before merging into the main branch.

Top Interview Questions — Architecture & Distributed Systems

Q1: Walk me through this project in simple software engineering terms.

Model Answer:

"I built a two-tier data engineering pipeline that processes biological data at both the individual sample and population cohort scale, surfacing insights through a live Streamlit dashboard.

Pipeline A takes binary alignment files (BAM) from a family dataset, deduplicates redundant reads using GATK, and calls genetic variants into structured VCF files.

Pipeline B handles population-scale data (~1.1M records from the 1000 Genomes project). Because this volume is too large for single-threaded processing, we convert the data into columnar Parquet format using ADAM on HDFS and run distributed SQL-style aggregations using PySpark.

Finally, our Streamlit UI connects to the MyVariant.info REST API to perform live ClinVar clinical lookups, dynamically classifying and styling pathogenic variants for medical triage. The core parsing logic is tested with 15 unit tests running on GitHub Actions CI."

Q2: Why did you use Apache Spark instead of Pandas for Pipeline B?

Model Answer:

"The core difference is **memory architecture and execution model**:

- **Pandas** loads the entire dataset into a single process's memory (RAM) and executes operations sequentially on a single CPU core. With ~1.1 million complex multi-allelic records, Pandas causes high memory pressure and CPU bottlenecks.
- **Apache Spark** uses a distributed, partitioned dataset model. It splits data across logical partitions and executes MapReduce operations (like groupBy, count, and sort) in parallel across all available CPU worker threads.
- Additionally, Spark integrates seamlessly with **Apache Parquet**, leveraging columnar compression and predicate pushdown to load only required columns, which is drastically faster than Pandas reading an entire CSV/VCF into memory."

Q3: What does local[*] mean in Spark, and how does it differ from a production cluster?

Model Answer:

"local[*]" instructs Spark to run in local single-node mode, where the asterisk (*) dynamically allocates as many worker threads as there are logical CPU cores on the host machine.

In a true production environment (such as AWS EMR, Databricks, or Google Cloud Dataproc), Spark uses a cluster manager (YARN, Kubernetes, or Spark Standalone). The **Driver Node** creates the execution DAG, while multiple distinct **Worker Nodes** communicate over a high-speed network. In local mode, inter-thread data transfer happens via shared RAM rather than network sockets."

Top Interview Questions — APIs, GATK & Trade-offs

Q4: How does the REST API integration work, and how did you prevent rate-limiting?

Model Answer:

"When a user triggers the ClinVar lookup in Streamlit:

1. The application constructs a standardized **HGVS query identifier**: `chr{chrom}:g.{pos}{ref}>{alt}`.
2. It dispatches an HTTP GET request to the public `MyVariant.info` REST endpoint with a 3-second timeout guard.
3. The returned JSON payload is parsed to safely extract clinical significance and condition descriptions, accounting for variable schemas (dicts vs lists).

To protect the free public API and prevent our IP from receiving **HTTP 429 (Too Many Requests)** status codes, I implemented client-side rate throttling using `time.sleep(0.5)` between iterative calls and bounded synchronous execution to the top 10 rows."

Q5: What is the purpose of Duplicate Marking before Variant Calling?

Model Answer:

"During NGS library preparation, DNA fragments undergo PCR amplification cycles. Occasionally, identical duplicates of the same original DNA fragment bind across multiple flow cell clusters.

If not removed, an artificial PCR error could be sequenced 30 times and falsely called as a high-confidence homozygous mutation. `MarkDuplicates` identifies read pairs with identical start/end alignment coordinates, keeps the read with the highest Phred quality score sum, and flags the others so downstream variant callers ignore them."

Q6: Why is GATK HaplotypeCaller better than traditional pileup callers (like Samtools mpileup)?

Model Answer:

"Traditional pileup callers examine each chromosomal position independently in isolation. When insertions or deletions (Indels) occur, alignment algorithms often place misaligned gap penalties, causing false SNP calls around the indel.

GATK HaplotypeCaller overcomes this by performing **local de novo assembly**: it throws away the initial alignment in active regions, constructs a De Bruijn graph from read k-mers, reconstructs full candidate haplotypes, and aligns them against the reference using PairHMM. This provides vastly superior accuracy in complex and repetitive genomic regions."

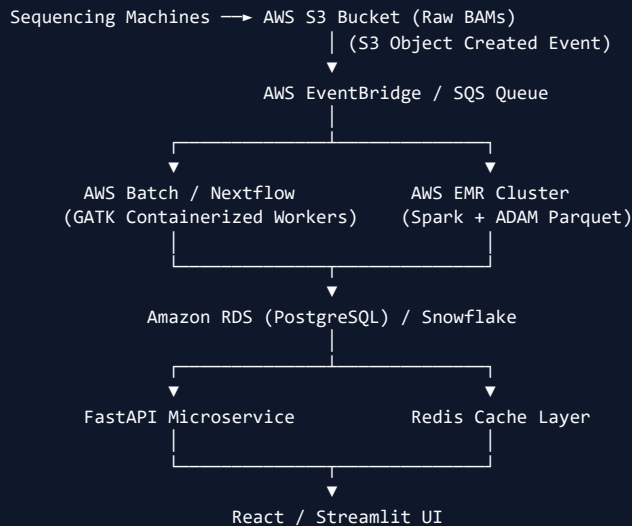


Production Scaling & System Design Questions

Q7: How would you scale this pipeline to process 10,000 whole human genomes per day?

Model Answer (System Design Perspective):

"To scale from a local demo to an enterprise-grade high-throughput genomics platform, I would re-architect the system into an **Event-Driven Cloud Architecture** on AWS:



- **Workflow Orchestration:** Use **Nextflow / WDL on AWS Batch** to parallelize GATK calling across thousands of spot EC2 instances.
- **Distributed Compute:** Deploy PySpark on **AWS EMR with S3 (EMRFS)** instead of local HDFS, taking advantage of auto-scaling worker nodes.
- **Caching Layer:** Store repetitive ClinVar REST lookups in a **Redis cache** to achieve sub-millisecond lookups and eliminate external API rate limits.
- **Data Warehousing:** Store called VCF variants in **Snowflake or Amazon Athena** using Parquet partitions partitioned by chromosome and sample_id for sub-second analytical querying."

Q8: If you had to add a Machine Learning model to this project, where would it fit?

Model Answer:

"Currently, the ClinVar integration is a **rule-based lookup** against existing medical literature. If a patient possesses a novel, unstudied mutation, ClinVar returns 'No Record Found'.

I would train a **Pathogenicity Prediction Classifier** (e.g., XGBoost or a Deep Learning variant effect model):

- **Feature Engineering:** Extract evolutionary conservation scores (PhyloP, PhastCons), chemical amino acid substitution matrices (BLOSUM62), protein structural disruption scores, and allele frequency data.
- **Training Target:** Train on known ClinVar labels ('Pathogenic' vs 'Benign').
- **Inference:** When a novel variant is detected, the model outputs a probability score predicting whether the mutation is likely disease-causing."

⚡ Final Interview Cheat Sheet & Key Facts

1. Vital Numbers & Facts to Memorize

Metric / Parameter	Exact Value	Context
Sample 1 Identity	NA12878 / HG001	Mother in CEPH Utah Pedigree 1463; NIST GIAB primary gold-standard reference.
Sample 2 Identity	NA12877	Father in CEPH Utah Pedigree 1463 (NA12878's husband).
Genomic Target Window	chr20:10M–10.2M (200 kbp)	Region of Human Chromosome 20 called by GATK HaplotypeCaller.
Pipeline B Cohort Size	~1.1 Million Variants	1000 Genomes Project Phase 3 Chromosome 22 cohort.
Benchmark Accuracy	Precision: 99.72% Recall: 99.15% F1: 99.44%	352 True Positives, 1 False Positive, 3 False Negatives against GIAB v4.2.1.
Unit Test Suite	15 PyTest Unit Tests	Runs automatically via GitHub Actions CI across Python 3.10 and 3.11.

2. 5 Golden Rules for Tomorrow's Interview

- 1. Frame as Software & Data Engineering:** Focus on Spark partitions, MapReduce, REST APIs, JSON parsing, PyTest, and CI/CD rather than biology. Presidio interviewers are software professionals.
- 2. Know the Difference Between Pipeline A & B:** Pipeline A = GATK sample calling on Chromosome 20; Pipeline B = Spark population statistics on 1.1M variants of Chromosome 22. They do not feed into each other directly; they unify at the Streamlit dashboard.
- 3. Be Honest About Limitations:** If asked about scale, openly mention that Spark ran in `local[*]` mode and GATK called a 200 kbp window, but explain how you would scale to AWS EMR and AWS Batch in production.
- 4. Highlight Code Quality:** Emphasize that you wrote 15 unit tests and configured a GitHub Actions matrix pipeline to ensure software reliability.
- 5. Use Structured Answer Formats:** State the problem → Explain your architectural choice → Mention trade-offs / outcomes.

3. Rapid Vocabulary Map

Bioinformatics Term	Software Engineering Translation to Use in Interview
BAM File	Binary compressed alignment data records.
VCF File	Structured tabular data file containing genomic coordinate anomalies.
Variant / Mutation	Data point deviation from the reference baseline schema.
HaplotypeCaller	Statistical inference engine running local graph assembly and HMM scoring.
ClinVar	Centralized public database linking genetic anomalies to medical conditions.
ADAM / Parquet	Columnar data serialization providing predicate pushdown and high compression.